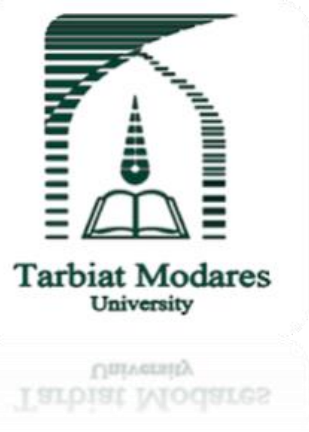




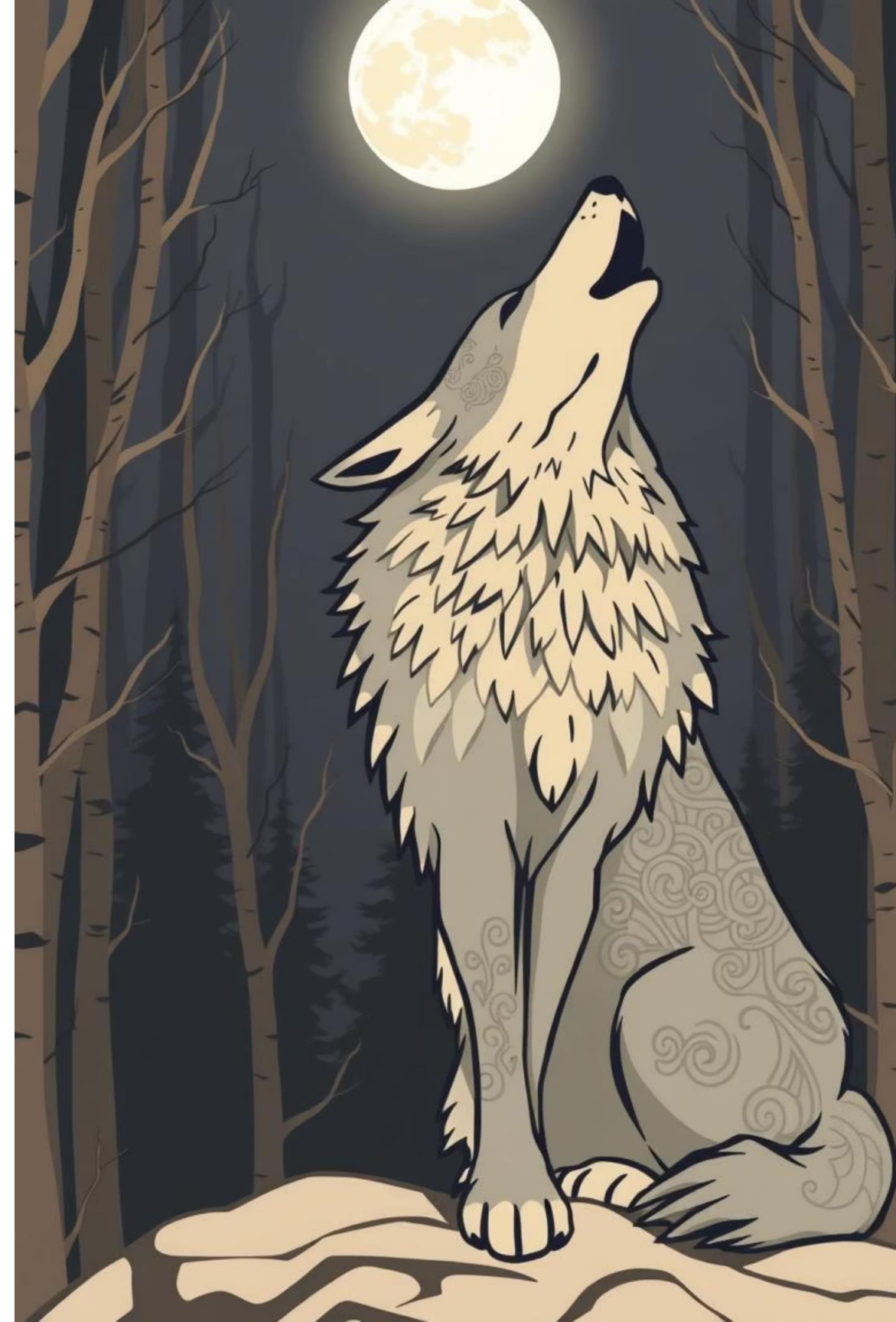
The Grey Wolf Optimizer

Advanced Algorithms

Dr. F. Z. SABERIFAR

By : Alireza Hosseinzade

Department of Mathematical Sciences, Tarbiat Modares University



Understanding Heuristics

Definition

- *In combinatorial optimization, a heuristic refers to a consistent algorithm based on a transparent and usually simple search strategy.*
- *The primary goal is to explore the set of all feasible solutions efficiently, especially when the search space is vast.*

The Optimality Gap

- *Unlike exact algorithms, heuristics do not guarantee finding the global optimal solution.*
- *They are designed to provide "good enough" solutions within a reasonable time frame for complex problems.*

Heuristics in a Narrow Sense

- *Techniques where rigorous mathematical proof of quality or polynomial time complexity is currently unavailable.*
- *Despite the lack of formal analysis, the underlying logic promises robust behavior for typical problem instances.*

Heuristic vs. Approximation

- *An algorithm is considered a heuristic while its behavior (complexity and solution quality) remains unanalyzed.*
- *Once reasonable mathematical bounds are proven, it transitions into a (Randomized) Approximation Algorithm.*

Search Mechanisms: Exploration vs. Exploitation

The Search Process

- **Meta-heuristic search is a dynamic balance between two contradictory phases: Exploration and Exploitation.**
- **The goal is to navigate the search space to find the global optimum while avoiding stagnation.**

Exploration

- **Definition: The ability of the algorithm to visit broad and diverse regions of the search space.**
- **Purpose: To prevent the algorithm from getting trapped in Local Optima (deceptive local peaks).**
- **Mechanism: Large-scale movements or random-like steps to discover new potential regions.**

Exploitation

- **Definition: The ability to focus the search on a local region that is already known to be promising.**
- **Purpose: To refine and improve the current best solution to reach the exact Global Optimum.**
- **Mechanism: Small, deliberate steps around the best-found candidate solutions.**

The Crucial Balance

- **Too much Exploration: The algorithm becomes a "Random Search" and may never converge.**
- **Too much Exploitation: The algorithm converges too early (Premature Convergence) to a sub-optimal point.**

Classification of Metaheuristic Algorithms

1

Evolutionary

- ***Genetic Algorithms (GA)***
- ***Evolution Strategies (ES)***
- ***Differential Evolution (DE)***
- ***Genetic Programming (GP)***

2

Bio-Inspired (Plant & Organism-Based)

- ***Flower Pollination Algorithm (FPA)***
- ***Plant Growth Simulation Algorithm (PGSA)***
- ***Invasive Weed Optimization (IWO)***
- ***Artificial Plant Optimization (APO)***

3

Physics-Based

- ***Simulated Annealing (SA)***
- ***Gravitational Search Algorithm (GSA)***
- ***Harmony Search (HS)***
- ***Black Hole Algorithm (BHA)***

4

Swarm Intelligence

- ***Particle Swarm Optimization (PSO)***
- ***Ant Colony Optimization (ACO)***
- ***Firefly Algorithm (FA)***
- ***Bat Algorithm (BA)***

Inspiration from Nature: The Grey Wolf Pack



Grey wolves are fascinating creatures that live in highly organised packs, governed by a strict social hierarchy. This hierarchy consists of:

- *Alpha: The leaders, responsible for decision-making.*
- *Beta: Subordinate wolves assisting the Alpha.*
- *Delta: Subordinates who dominate the Omega wolves.*
- *Omega: The lowest-ranking members.*

Their cooperative hunting strategies, involving tracking, encircling, and attacking prey, provide a perfect blueprint for optimization. This intricate natural behaviour directly inspired the creation of the Grey Wolf Optimizer algorithm.

The Grey Wolf Optimizer (GWO) Algorithm Overview

Classification: Swarm Intelligence

The Grey Wolf Optimizer is classified as a Swarm Intelligence-based metaheuristic algorithm, as it mimics the collective behavior and social structure of wolf packs to solve optimization problems.

1

Population-Based

A metaheuristic approach that meticulously mimics the social structure and hunting behaviour of grey wolf packs.

2

Wolf Roles

Each wolf in the algorithm represents a potential solution within the complex search space. The Alpha, Beta, and Delta wolves act as the primary guides, leading the search process.

3

Omega Followers

Omega wolves dutifully follow, updating their positions based on the insights provided by the leading wolves.

4

Three Phases

The algorithm progresses through distinct phases: encircling the prey, initiating the hunt, and finally attacking to converge on the optimal solution.

How GWO Works: Mathematical Model Simplified

1

Position Updates

The core of GWO lies in how each wolf updates its position, influenced by the vectors determined by the Alpha, Beta, and Delta wolves, reflecting their guidance.

2

Exploration & Exploitation

Carefully calibrated coefficients govern the balance between exploration (searching new solution areas) and exploitation (fine-tuning existing promising solutions).

3

Dynamic Parameter 'a'

A crucial parameter, 'a', linearly decreases from 2 to 0 over the course of iterations, gradually shifting the algorithm's focus from broad exploration to precise exploitation.

$$A = 2a \cdot r_1 - a$$

4

Simulated Hunt

Wolves continually update their positions by metaphorically encircling the best solutions found thus far, effectively simulating the cooperative hunting process.

$$X(t+1) = X_p(t) - A \cdot |C \cdot X_p(t) - X(t)|, \text{ where } C = 2 \cdot r_2$$

Where X is the wolf's position, X_p is the prey's position, t is the iteration, and A and C are coefficient vectors.

Social Hunting Mechanism: Position Updating

Leadership Guidance

- *In GWO, we assume that the Alpha, Beta, and Delta have better knowledge about the potential location of the prey.*
- *The first three best solutions obtained so far are saved, and the other search agents (Omegas) are obliged to update their positions according to the best search agents.*

Step 1: Calculating the distance from the leaders.

$$\vec{D}_\alpha = | \vec{C}_1 \vec{X}_\alpha - \vec{X} |$$

$$\vec{D}_\beta = | \vec{C}_2 \vec{X}_\beta - \vec{X} |$$

$$\vec{D}_\delta = | \vec{C}_3 \vec{X}_\delta - \vec{X} |$$

Step 2: Determining the direction of movement.

$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 (\vec{D}_\alpha)$$

$$\vec{X}_2 = \vec{X}_\beta - \vec{A}_2 (\vec{D}_\beta)$$

$$\vec{X}_3 = \vec{X}_\delta - \vec{A}_3 (\vec{D}_\delta)$$

Step 3: Final Position Update (Averaging)

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3}$$

Grey Wolf Optimizer (GWO) Pseudocode

1. Initialize the population of n grey wolves (X_i)
2. Initialize parameters a , A , and C :
 - a : decreases linearly from 2 to 0
 - $A = 2a * r_1 - a$
 - $C = 2 * r_2$
 - (r_1, r_2 are random vectors in $[0, 1]$)
3. Calculate fitness for each wolf:
 - X_{α} : Best solution
 - X_{β} : Second best
 - X_{δ} : Third best
4. While ($t < \text{Max_iterations}$)
 - For each wolf X_i :
 - // Step 1: Calculate distance from leaders (Slide 7)
 - $D_{\alpha} = |C_1 * X_{\alpha} - X_i|$
 - $D_{\beta} = |C_2 * X_{\beta} - X_i|$
 - $D_{\delta} = |C_3 * X_{\delta} - X_i|$

- // Step 2: Calculate step size towards leaders
 - $X_1 = X_{\alpha} - A_1 * D_{\alpha}$
 - $X_2 = X_{\beta} - A_2 * D_{\beta}$
 - $X_3 = X_{\delta} - A_3 * D_{\delta}$
 - // Step 3: Update current wolf position (Slide 8)
 - $X_i(t+1) = (X_1 + X_2 + X_3) / 3$
 - End For
 - Update a , A , and C
 - Evaluate fitness of all wolves
 - Update X_{α} , X_{β} , and X_{δ}
 - $t = t + 1$
 - End While
5. Return X_{α}

Case Study Definition

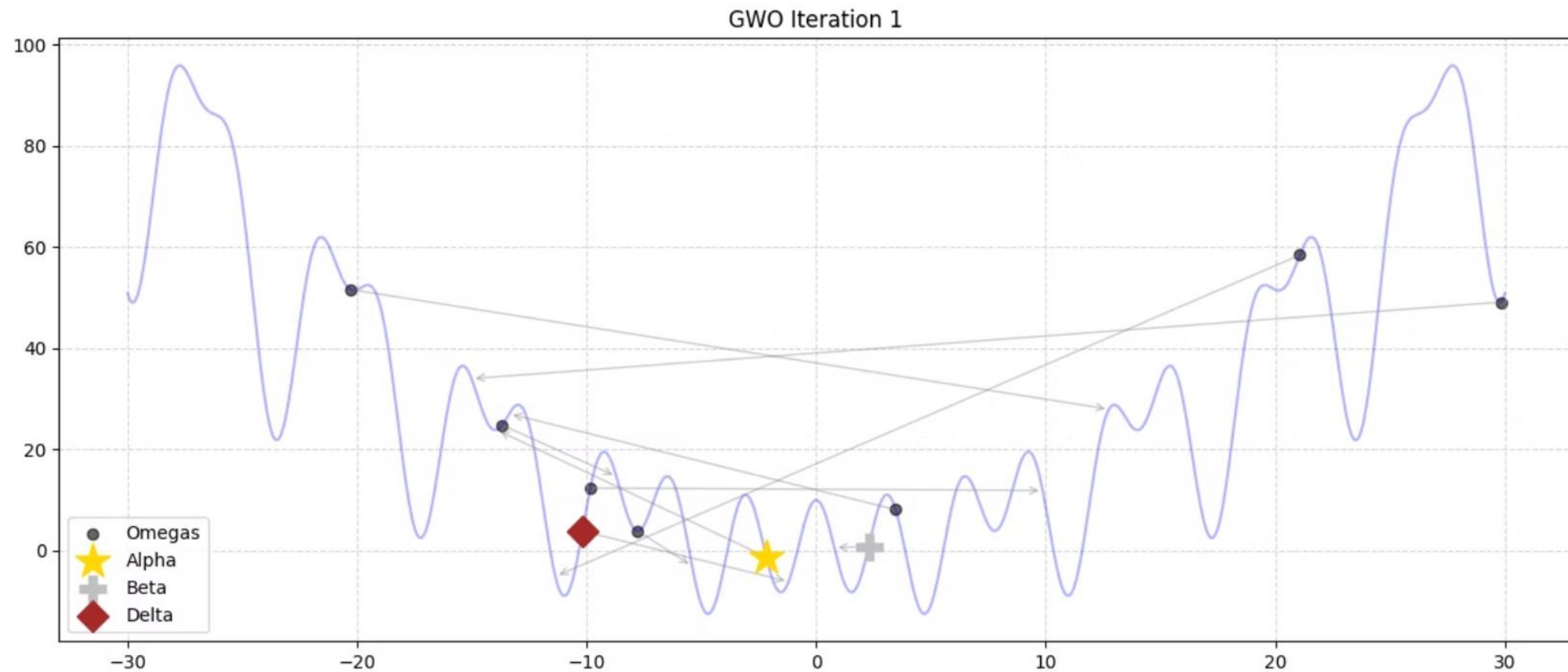
- ***Objective Function:***

$$f(x) = x \sin(x) + 0.1x^2 + 10 \cos(2x)$$

- ***Search Space:***

$$x \in [-30, 30]$$

- ***Optimization Goal: Finding the Global Minimum (The deepest valley).***
- ***Simulation Setup:***
- ***Population Size: 10 Wolves***
- ***Max Iterations: 16***
- ***Challenge: The function has multiple local optima, testing the algorithm's ability to avoid traps.***



- ***Initial Population: 10 search agents (wolves) randomly distributed using Uniform Distribution.***
- ***Landscape Complexity: Multiple local minima that can trap a standard optimizer.***
- ***Status at $t=1$: The algorithm identifies the top 3 performers (alpha, beta, delta) to lead the hunt.***

Numerical Trace – Search Agent Update

Iteration 1 (Grey Wolf Optimizer)

1. Initial State & Leader Wolves

Target Agent (Wolf #7):

$$X(t) = 21.0647$$

Leader Positions:

- Alpha (α): $X_{\alpha} = -2.1775$
- Beta (β): $X_{\beta} = 2.2857$
- Delta (δ): $X_{\delta} = -10.1850$

Control Parameter:

$$a = 2.0000$$

2. Leader Influence Calculations

Alpha (α)

$$C_{\alpha} = 0.1377, A_{\alpha} = 0.1601$$

$$D_{\alpha} = |(C_{\alpha} \cdot X_{\alpha}) - X(t)| = |(0.1377 \cdot -2.1775) - 21.0647| = 21.3645$$

$$X_1 = X_{\alpha} - A_{\alpha} \cdot D_{\alpha} = -2.1775 - (0.1601 \cdot 21.3645) = -\mathbf{5.5979}$$

Beta (β)

$$C_{\beta} = 0.5690, A_{\beta} = 1.1315$$

$$D_{\beta} = |(C_{\beta} \cdot X_{\beta}) - X(t)| = |(0.5690 \cdot 2.2857) - 21.0647| = 19.7640$$

$$X_2 = X_{\beta} - A_{\beta} \cdot D_{\beta} = 2.2857 - (1.1315 \cdot 19.7640) = -\mathbf{20.0765}$$

Delta (δ)

$$C_{\delta} = 1.6217, A_{\delta} = -0.0470$$

$$D_{\delta} = |(C_{\delta} \cdot X_{\delta}) - X(t)| = |(1.6217 \cdot -10.1850) - 21.0647| = 37.5813$$

$$X_3 = X_{\delta} - A_{\delta} \cdot D_{\delta} = -10.1850 - (-0.0470 \cdot 37.5813) = -\mathbf{8.4191}$$

3. Final Position Update

Update Equation:

$$X(t + 1) = \frac{X_1 + X_2 + X_3}{3}$$

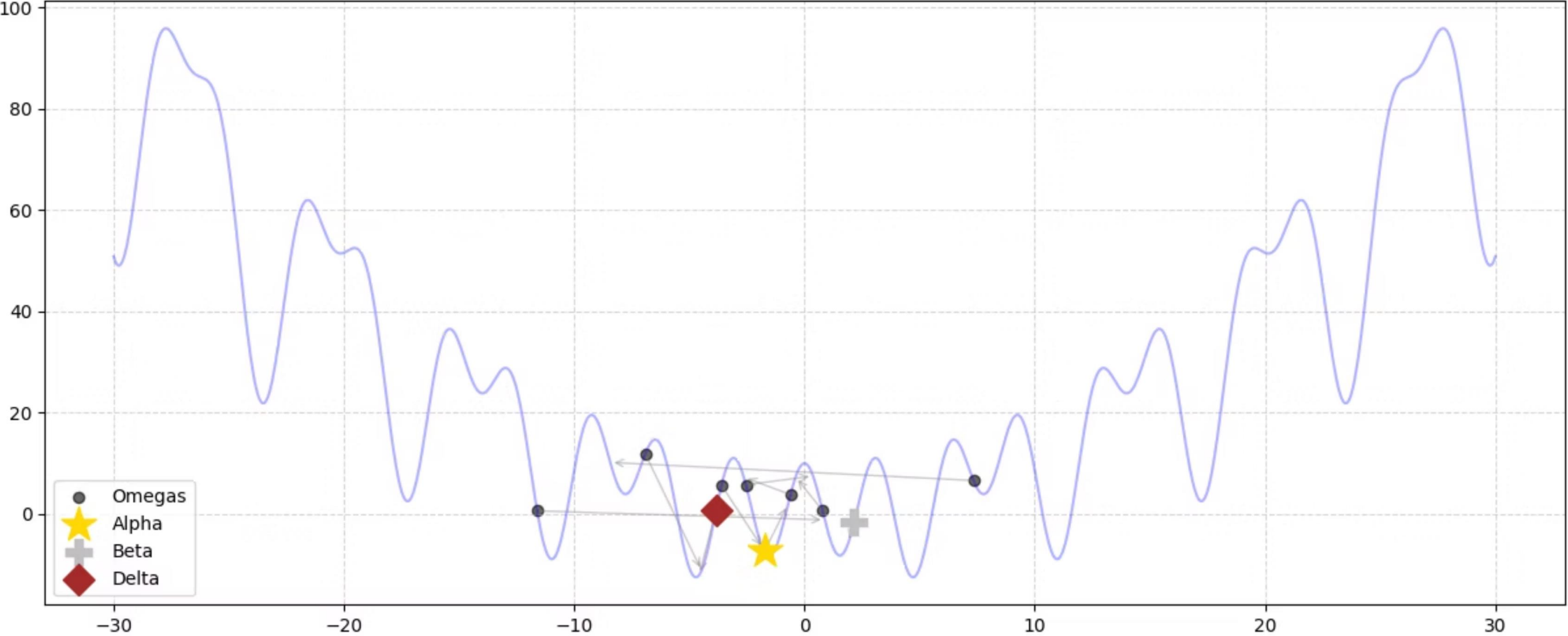
Substitution:

$$X(t + 1) = \frac{-5.5979 - 20.0765 - 8.4191}{3}$$

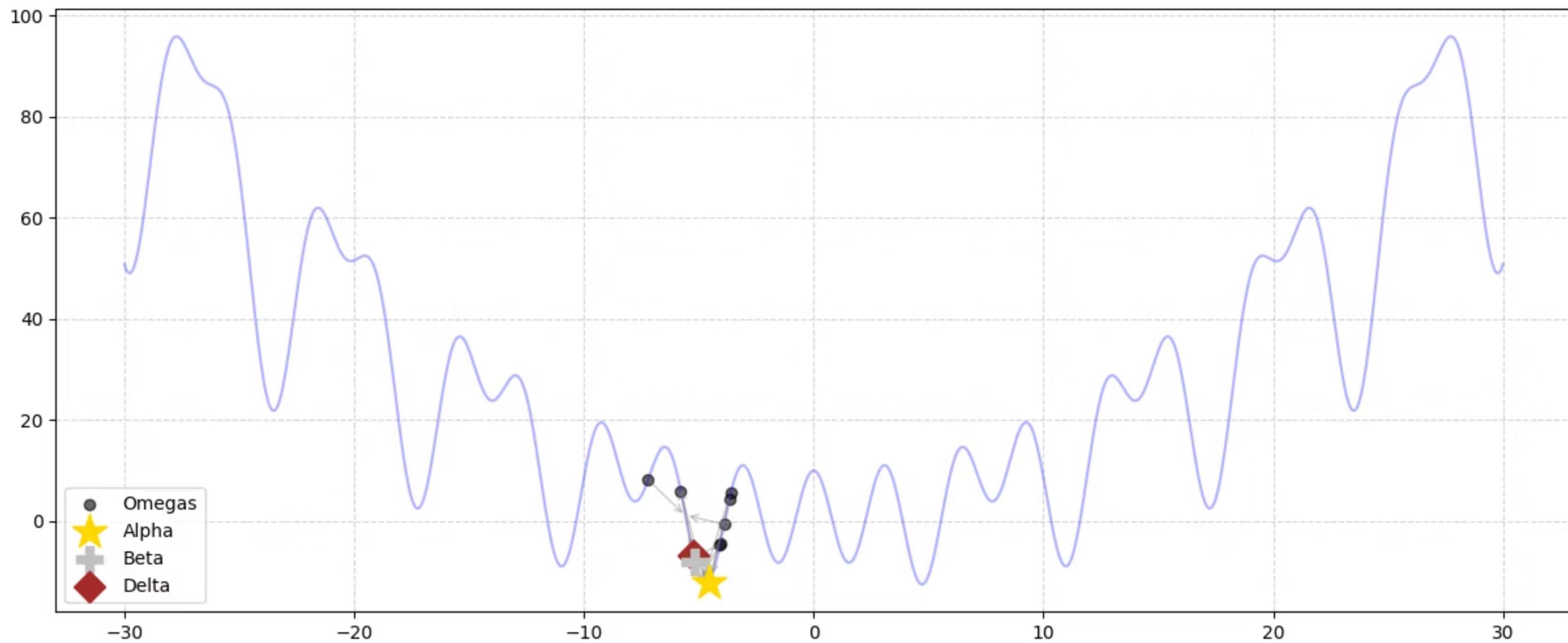
New Agent Position:

$$\boxed{X(t + 1) = -11.3645}$$

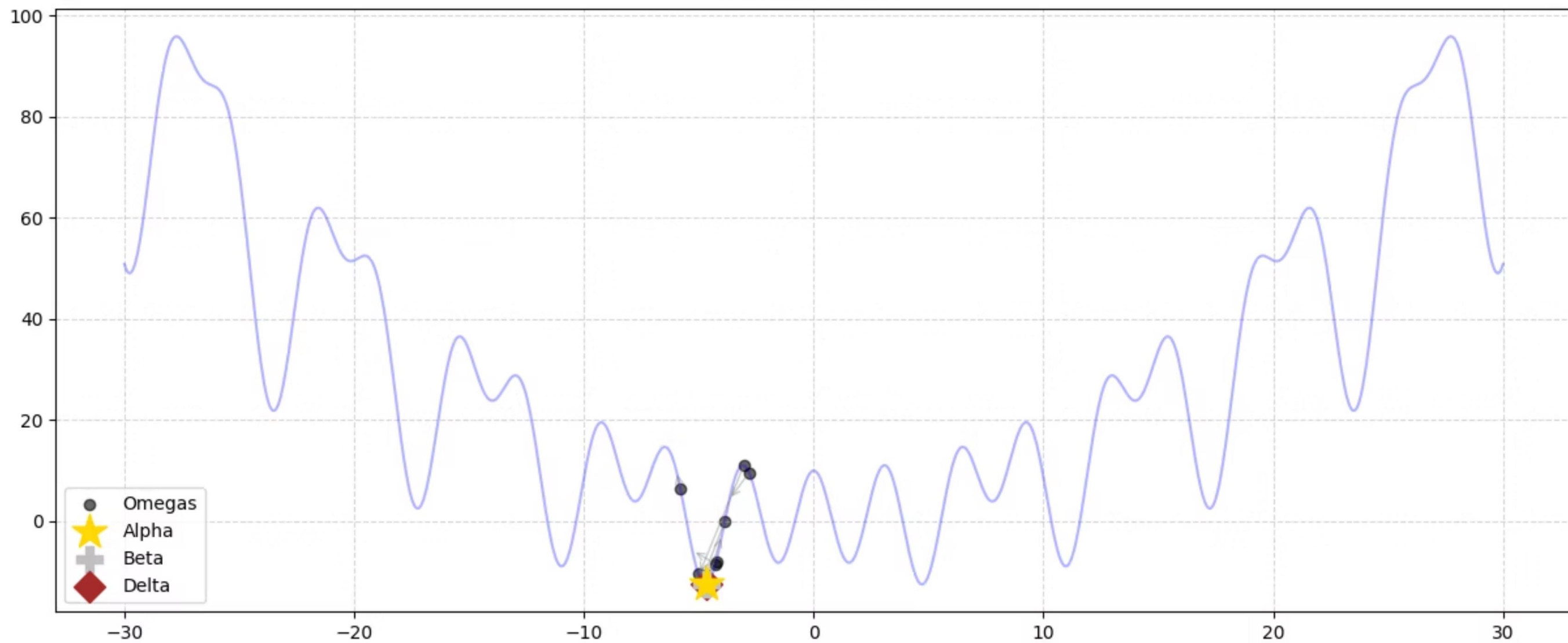
GWO Iteration 4



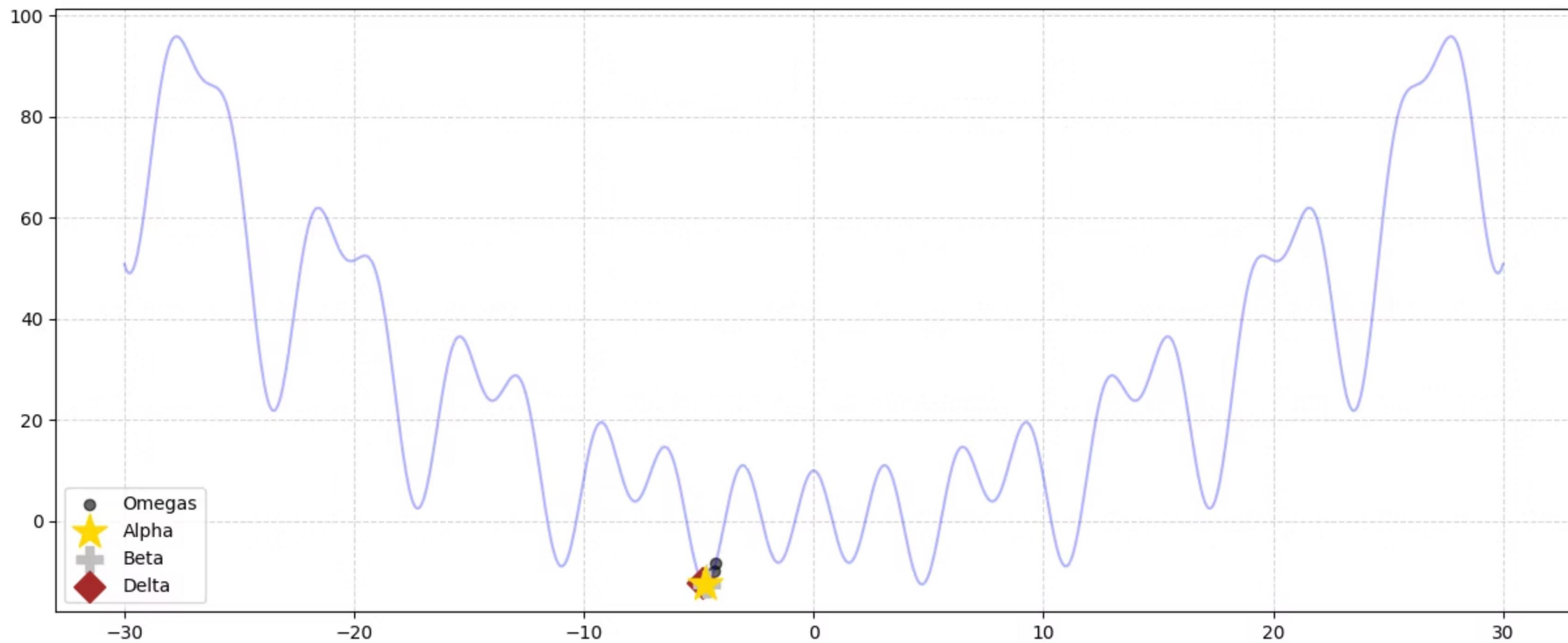
GWO Iteration 8



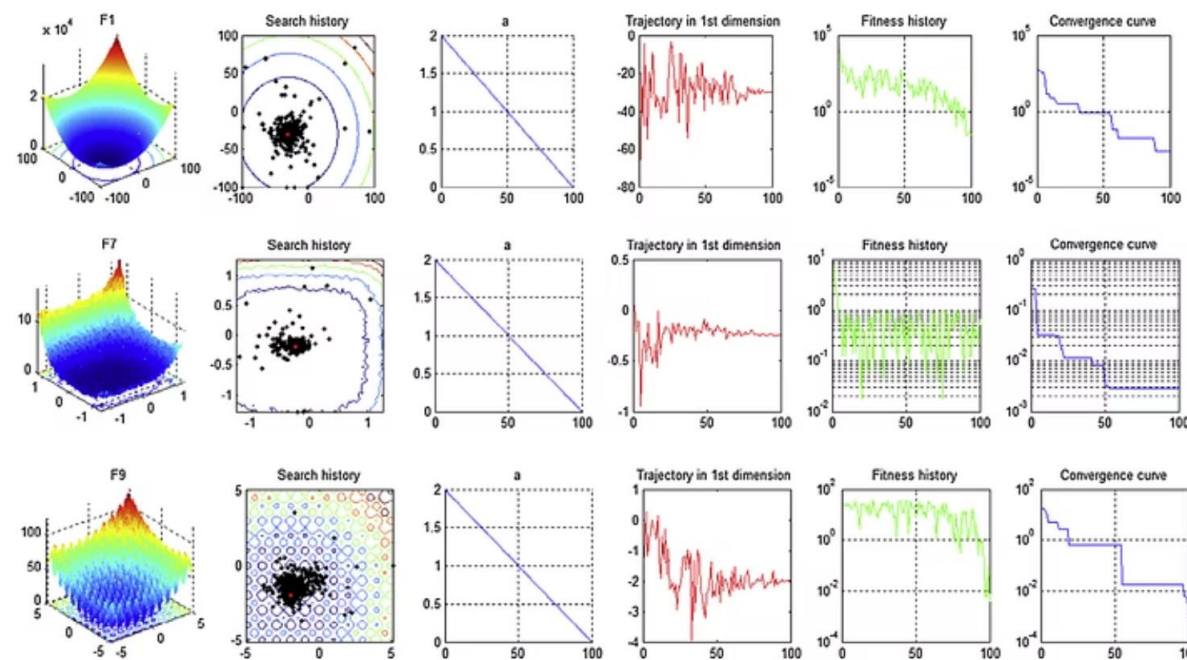
GWO Iteration 12



GWO Iteration 16



Visualising GWO in Action



These visuals depict the dynamic process of position updates, illustrating how wolves (solutions) progressively move closer to the prey (optimal solution). The convergence curve further highlights GWO's efficiency in reaching optimal solutions compared to other algorithms.

Advantages & Challenges of GWO

Advantages

Simple to implement with minimal parameters.

Effective balance of exploration and exploitation capabilities.

Good convergence speed and strong global search ability.

Challenges

Can sometimes get trapped in local optima in highly complex problems.

Performance can be sensitive to initial population and parameter tuning.



Recent Advances

→ Improved Variants

Refreshing mechanisms

New operators

Modified population structure and hierarchy

→ Parallelism

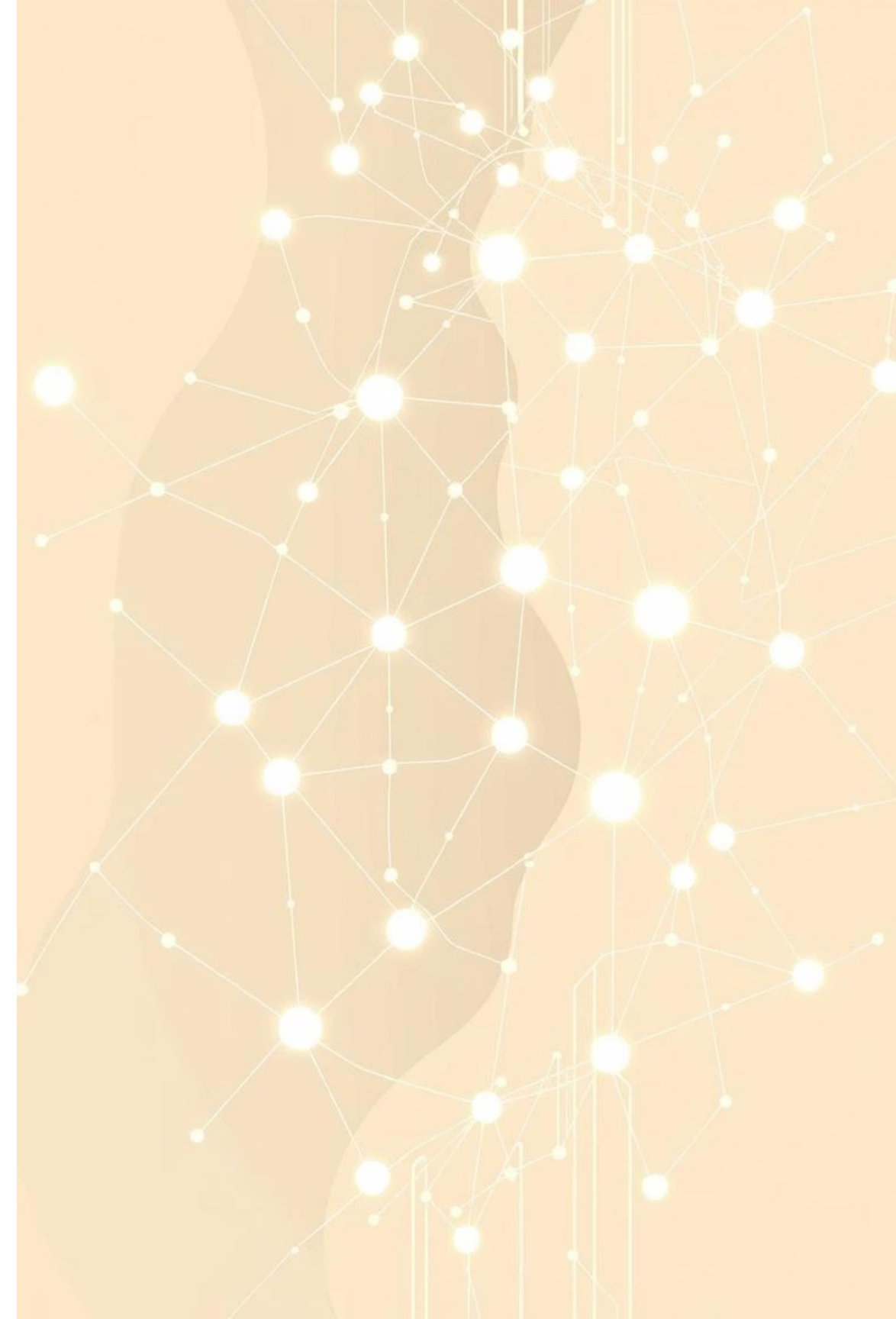
In population-based meta-heuristics, parallelism is a useful strategy that allows the population to be split up into several subpopulations, each of which is capable of evolving on a different type of machine processor.

→ Hybrid Algorithms

Innovative hybrid algorithms combine GWO with other powerful techniques, such as neural networks, fuzzy logic, or differential evolution, leveraging their combined strengths.

→ Multi-objective GWO

Unlike standard GWO that finds a single best solution, MOGWO aims to find a set of optimal trade-off solutions called the Pareto Optimal Set.

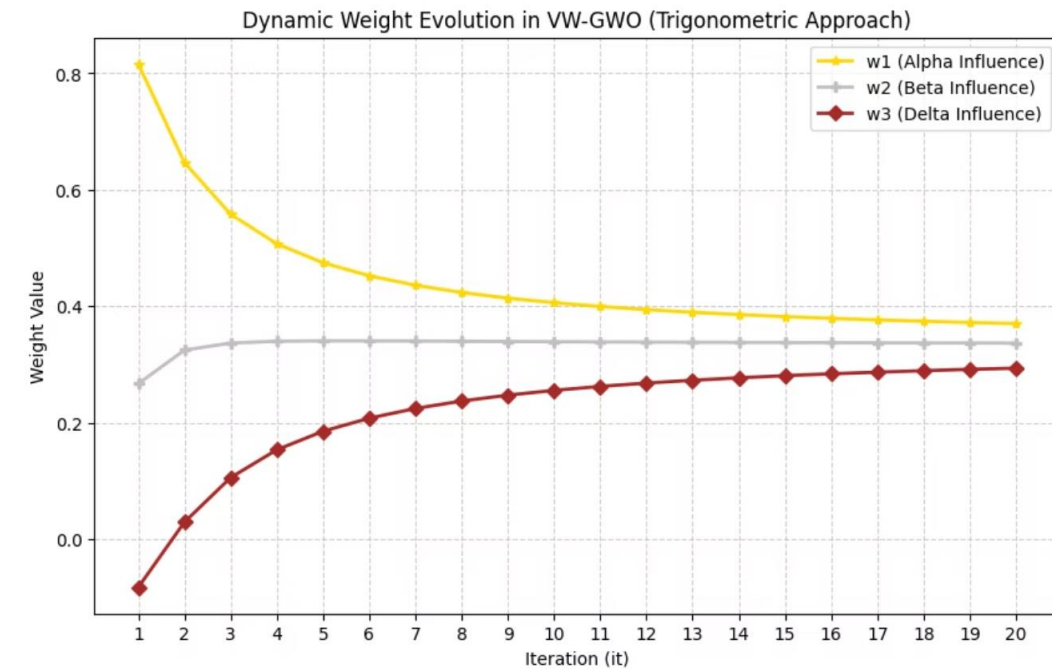


VW-GWO (Variable Weight Grey Wolf Optimizer)

In VW-GWO, the leadership roles of alpha, beta, and delta wolves are not assumed to be equally important.

Dynamic and iteration-dependent weights are assigned to the leaders, allowing the algorithm to better reflect the natural hierarchy of grey wolves and to achieve a more effective balance between exploration and exploitation.

$$\left\{ \begin{array}{l} \vec{X}(t+1) = \omega_1 \vec{X}_1 + \omega_2 \vec{X}_2 + \omega_3 \vec{X}_3 \\ \omega_1 + \omega_2 + \omega_3 = 1 \\ \varphi = \frac{1}{2} \arctan(it) \\ \theta = \frac{2}{\pi} \arccos \frac{1}{3} \cdot \arctan(it) \\ \omega_1 = \cos \theta \\ \omega_2 = \frac{1}{2} \sin \theta \cdot \cos \varphi \\ \omega_3 = 1 - \omega_1 - \omega_2 \end{array} \right.$$



IAGWO (Improved Alpha-Guided Grey Wolf Optimizer)

IAGWO enhances the influence of the alpha wolf as the best current solution.

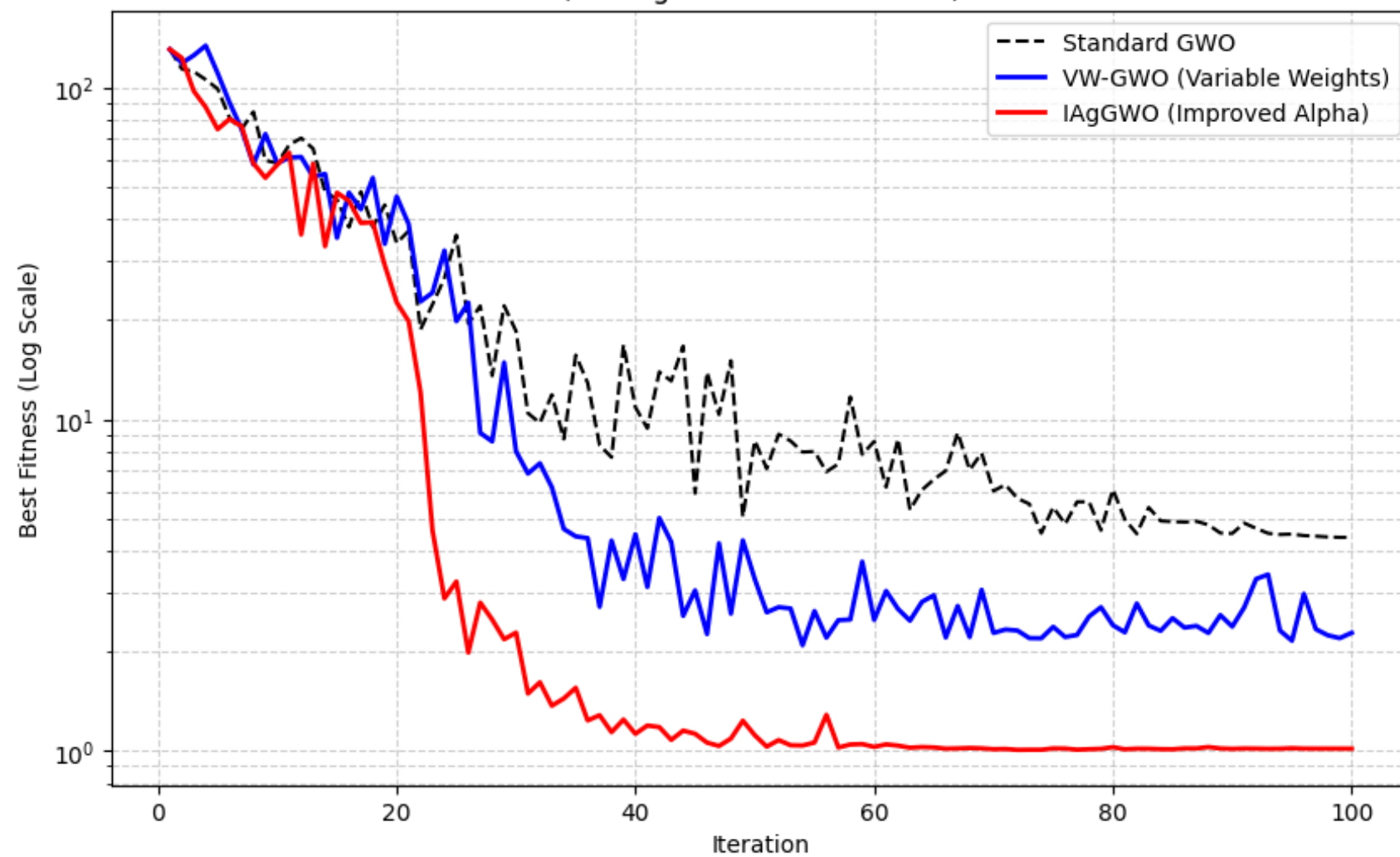
After the standard position update, Opposition-Based Learning and Gaussian mutation are applied only to the alpha wolf in order to increase population diversity and reduce the risk of premature convergence.

$$\vec{X}(t+1) = 0.5 \vec{X}_1(t) + 0.3 \vec{X}_2(t) + 0.2 \vec{X}_3(t)$$

$$\begin{cases} \vec{X}_{\alpha}^* = \vec{X}_{\max} + \vec{X}_{\min} - \vec{X}_{\alpha} \\ \vec{X}_{\alpha}^{**} = \text{Guassian}(1, 1) \times \vec{X}_{\alpha} \end{cases}$$

$$f(x) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$$

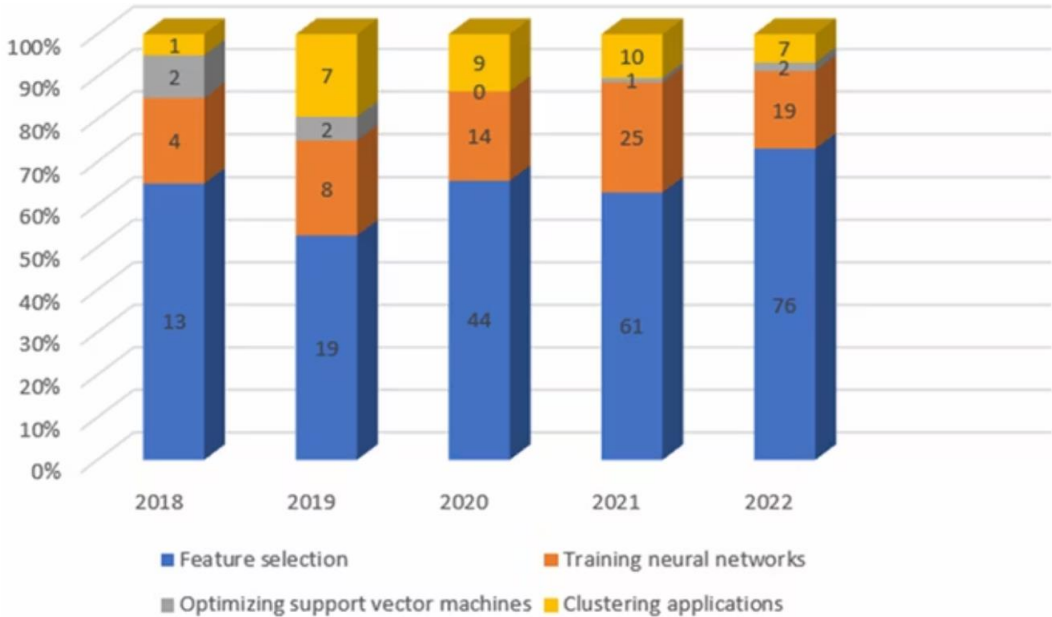
Convergence Comparison: Standard GWO vs Improved Variants
(Rastrigin Function - 10 Dim)



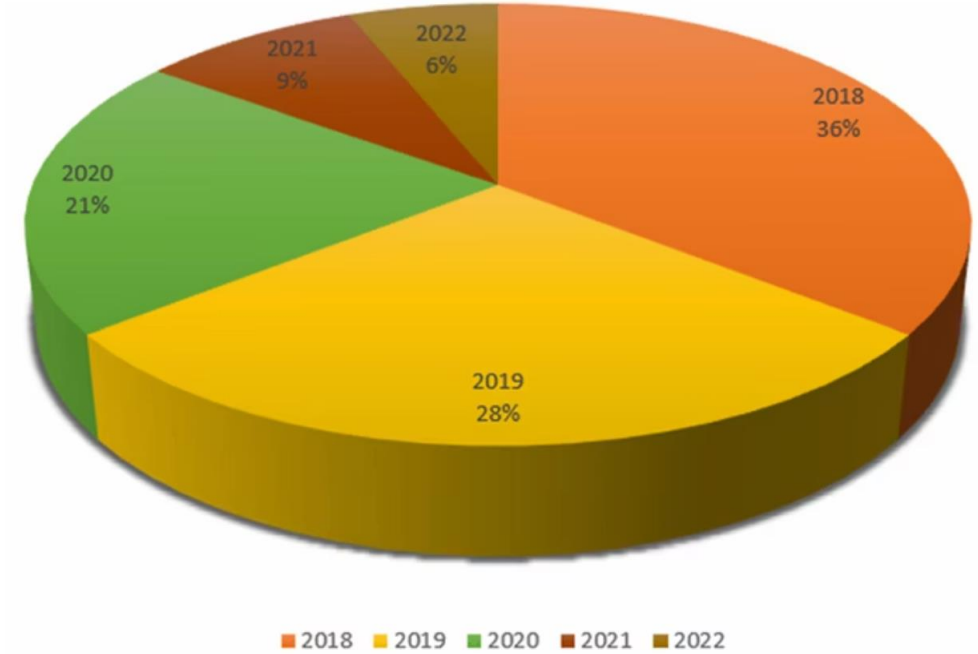
Real-World Applications of GWO

- Machine learning applications
- Feature selection*
- Training neural networks*
- Optimizing support vector machine*
- Clustering applications*
- Image processing*

- Engineering applications
- Design and tuning controllers*
- Power dispatch problems*
- Robotics and path planning*
- Scheduling*
- Wireless sensor network*



Publications using the GWO in machine learning



Publications using the GWO in feature selection

END

